Portfolio Optimization

Introduction

The datasets provided contained stock price of US equities from 1993-2019, however, I only looked at the visuals of portfolios in periods of 10 years. This was done using Mathematical Optimization Algorithm, Monte Carlo Simulation as well as the Modern Portfolio Theory. The visuals were created using Riskfolio-Lib. This report will look at the various methods used in exploratory data analysis and data wrangling and visualization, in order to assess the portfolios that maximize returns for the risk assumed.

Exploratory Data Analysis

The analysis was done in a systematic order where I dropped the empty rows and the rows outside the time being analyzed, and copied it to stock_short as these columns would not be in S&P 500. Then we Store the columns names, and calculate the return as (FinalPrice-StartPrice)/StartPrice, and the volatility as StandardDeviation/StartPrice. The Sharpe Ratio should have some no risk return to compare this against as it gives an indication of the performance of the stock over the period of interest.

After doing this, I plotted the volatility against the return for visual purposes along with getting a reference line to give and indication of he best performing stocks. It is worth noting that the best performing stocks are in contact with line through the origin with the largest slope.

Data Wrangling

Following the completion of exploratory data analysis, I decided to compute the returns and statistics necessary to run both the Monte Carlo Simulation and the Mathematical Optimization Algorithm. However, due to the size of the dataset, we created a smaller dataset named data1 with the 5 best performing stocks. I conducted data wrangling on both data frames concurrently as the results helped me determine which algorithm is suitable for the job.

This was done by getting the daily returns for each stock using the pct_change() method. This was my preferred method as they have the material of being asset-additive which in turn is needed to compute portfolio returns. Subsequently, the daily returns were annualized using .mean() method whilst assuming the number of trading days are 252.

```
# returns_portfolio stats
mean_returns = returns_portfolio.mean() * 252
covar = returns_portfolio.cov() * 252
correlation = returns_portfolio.corr() * 252
rf = 0
```

Figure 1: stats for retruns_ portfolio

```
# data1 stats
returns = data1.pct_change()
mean_returns = returns.mean()
cov_matrix = returns.cov()
num_portfolios = 25000
risk_free_rate = 0.0278
```

Figure 2: stats for data1

The risk factor in returns_portfolio is set to 0 because it will use Monte Carlo simulation whereas data1 will apply the Mathematical Optimization Algorithm. Thus, using a risk-free rate of 2.78% in the 52-week treasury bill rate of 2011.

Random Portfolio Generation.

Generating random portfolios was quite challenging as I had to find a method that spreads the capital provided between the assets in a manner that ensures maximum return of the portfolio. It is worth noting, there is no specific solution for this problem however, there are a cluster of solutions known as the efficient frontier. I tackled this task using 3 models:

- The Modern Portfolio Theory
- The Monte Carlo Simulation
- The Mathematical Optimization Algorithm

The Modern Portfolio Theory

The Markowitz model for the solution of the portfolio optimization problem has a twin objective of maximizing return and minimizing risk, built on the Mean-Variance framework of asset returns and holding the basic constraints.

I applied the Modern Portfolio Theory in order to get the efficient frontier. In favor of plotting the graph of efficient frontier using Markowitz Portfolio Optimization, I ran a loop. In each iteration, the loop considers different weights for assets and calculates the return and volatility of that particular portfolio combination. This loop runs n times.

```
[48] p_ret = [] # Define an empty array for portfolio returns
     p_vol = [] # Define an empty array for portfolio volatility
     p_weights = [] # Define an empty array for asset weights

     num_assets = len(psy.columns)
     num_portfolios = 20000

     for p in range(num_portfolios):
         weights = np.random.random(num_assets)
         weights = weights/np.sum(weights)
         p_weights.append(weights)
         returns = np.dot(weights, yearly_returns) # Returns are the product of individual expected returns of asset and its
                                                   # weights
         p_ret.append(returns)
         var = cov_matrix.mul(weights, axis=0).mul(weights, axis=1).sum().sum()# Portfolio Variance
         sd = np.sqrt(var) # Daily standard deviation
         ann_sd = sd*np.sqrt(253) # Annual standard deviation = volatility
         p_vol.append(ann_sd)
```

Figure 3: Markowitz optimization code

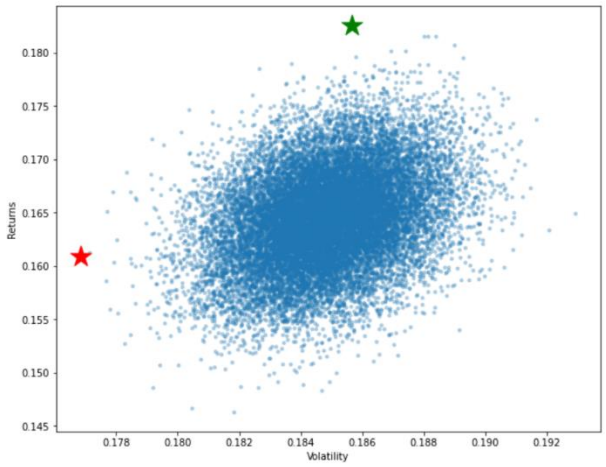


Figure 4: Effective frontier with minimum volatility and maximum sharpe ratio

The red star denotes the minimum volatility while the green star denotes the optimal risk portfolio.

The Monte Carlo Simulation

While applying this simulation, the first thing I did was mimic a random set of portfolios to visualize the risk-return profiles of our given set of assets. Consequently, I defined 2 functions that take inputs of asset weights and output the expected portfolio return and, standard deviation.

```
[74] #  daily returns = returns_portfolio
     # Function for computing portfolio return
     def portfolio_returns(weights):
       port = (np.sum(returns_portfolio.mean() * weights)) * 252
       return port

[75] # returns_portfolio

[76] #  Standard Deviation of the portfolio returns
     # Function for computing standard deviation of portfolio returns
     def portfolio_sd(weights):
       sdv = np.sqrt(np.transpose(weights) @ (returns_portfolio.cov() * 252) @ weights)
       return sdv
```

Figure 5: expected return function and standard deviation

A for loop was then created to simulate the random vectors of asset weights by processing the expected portfolio and standard deviation for each combination of weights. This action occurs randomly. It Is worth noting that the Monte Carlo simulation is computationally intensive due to the size of the dataset hence, making us settle for the mathematical optimization.

```python
# instantiate empty array containers for returns and sd
list_portfolio_returns = []
list_portfolio_sd = []
p_weights = [] # Define an empty array for asset weights

# For loop to simulate 2000 random weight vectors (numpy array objects)
for p in range(1000):
  # Return random floats in the half-open interval [0.0, 1.0)
  weights = np.array(np.random.random(symbols))
  # Normalize to unity
  # The /= operator divides the array by the sum of the array and rebinds "weights" to the new object
  weights /= np.sum(weights)
  p_weights.append(weights)

  # Lists are mutable so growing will not be memory inefficient
  list_portfolio_returns.append(portfolio_returns(weights))
  list_portfolio_sd.append(portfolio_sd(weights)) # do append and calculation separate

  # Convert list to numpy arrays
  port_returns = np.array(object = list_portfolio_returns)
  port_sd = np.array(object = list_portfolio_sd)

  sharpe_ratio = port_returns/port_sd
```

Figure 6: Monte Carlo Simulation code

The Mathematical Optimization Algorithm

This algorithm uses the optimization functions such as SciPy library to find the optimal weights mathematically. SciPy's in-built optimization algorithm creates an optimizer that attempts to minimize the negative Sharpe ratio. In turn, this helps calculate the weight allocation for the portfolio. This is seen in the function below.

```python
def neg_sharpe_ratio(weights, mean_returns, cov_matrix, risk_free_rate):
    p_var, p_ret = portfolio_annualised_performance(weights, mean_returns, cov_matrix)
    return -(p_ret - risk_free_rate) / p_var

def max_sharpe_ratio(mean_returns, cov_matrix, risk_free_rate):
    num_assets = len(mean_returns)
    args = (mean_returns, cov_matrix, risk_free_rate)
    constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
    bound = (0.0,1.0)
    bounds = tuple(bound for asset in range(num_assets))
    result = sco.minimize(neg_sharpe_ratio, num_assets*[1./num_assets,], args=args,
                          method='SLSQP', bounds=bounds, constraints=constraints)
    return result
```

Figure 7: Maximum Sharpe Ratio Portfolio

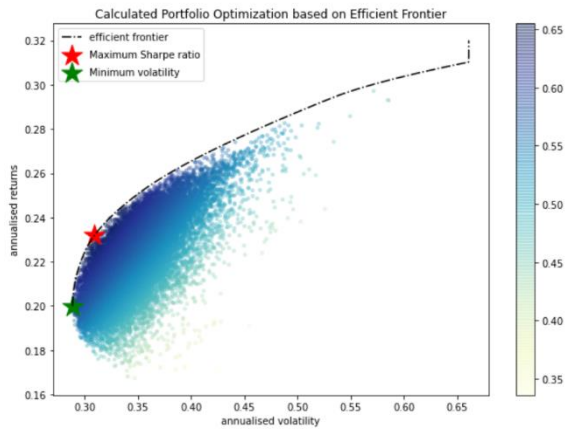
Since there is no maximize in the SciPy library, the function passes something that is minimized hence the neg_sharp_ratio. SciPy's minimize function can be used to generate portfolios that will also plot the efficient frontier as seen below.

```python
def display_calculated_ef_with_random(mean_returns, cov_matrix, num_portfolios, risk_free_rate):
    results, _ = random_portfolios(num_portfolios,mean_returns, cov_matrix, risk_free_rate)

    max_sharpe = max_sharpe_ratio(mean_returns, cov_matrix, risk_free_rate)
    sdp, rp = portfolio_annualised_performance(max_sharpe['x'], mean_returns, cov_matrix)
    max_sharpe_allocation = pd.DataFrame(max_sharpe.x,index=data1.columns,columns=['allocation'])
    max_sharpe_allocation.allocation = [round(i*100,2)for i in max_sharpe_allocation.allocation]
    max_sharpe_allocation = max_sharpe_allocation.T

    min_vol = min_variance(mean_returns, cov_matrix)
    sdp_min, rp_min = portfolio_annualised_performance(min_vol['x'], mean_returns, cov_matrix)
    min_vol_allocation = pd.DataFrame(min_vol.x,index=data1.columns,columns=['allocation'])
    min_vol_allocation.allocation = [round(i*100,2)for i in min_vol_allocation.allocation]
    min_vol_allocation = min_vol_allocation.T
```

Figure 8: SciPy's minimize function in use




```
-----------------------------------------------------------------------------
Maximum Sharpe Ratio Portfolio Allocation

Annualised Return: 0.23
Annualised Volatility: 0.31


            AMG US Equity  CCK US Equity  BLK US Equity  TEN US Equity  \
allocation            0.0          10.88          57.57          16.43

            REGN US Equity
allocation           15.13
-----------------------------------------------------------------------------
Minimum Volatility Portfolio Allocation

Annualised Return: 0.2
Annualised Volatility: 0.29
```

Figure 9: Efficient Frontier using SciPy's

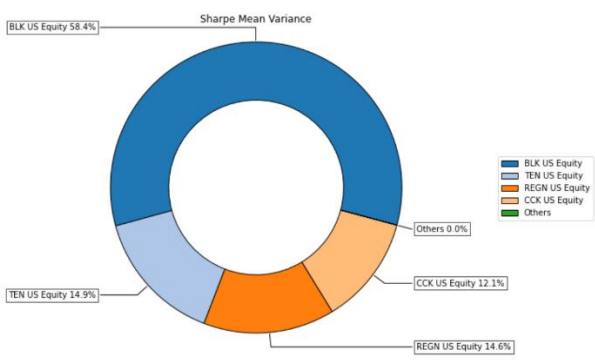
Figure 10: Portfolio Allocation

We have achieved relatively similar results with the randomly generated portfolios; however, this is simulated. The slight difference is that the Scipy's "optimize" function has not allocated any budget for Affiliated Managers Group (AMG) on maximum Sharpe ratio portfolio, while, one we chose from the randomly generated samples has 0.24% of allocation for Affiliated Managers Group.

Data Visualization of the efficient frontiers.

Extensive data visualization was only done on the smaller dataset as the original dataset was too big so, it would have resulted in overloaded plots.

Riskfolio-Lib calculates optimum portfolios and quantitative strategic asset allocation. This in turn results from optimization of one of the following functions: Maximum Return Portfolio, Minimum Risk Portfolio, Maximum Utility Portfolio and Maximum Risk Adjusted Return Ratio Portfolio. I used this library to compose and organize the portfolios and assets.



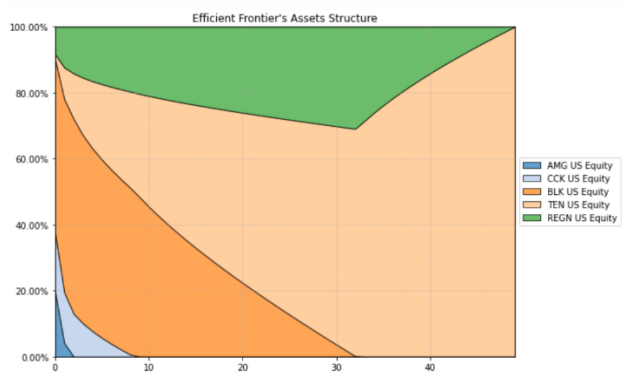


Figure 11:Portfolio Composition

Figure 12: Assets structure

BLK US Equity takes up the most in the portfolio composition similarly to figure 10. However, it takes up the least in Asset structure, it is not known why.